

Project 01 — Travel Guide

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overview

A “Travel Guide” website that helps visitors from different countries get suggestions and preferences for selecting visiting places worldwide. The website is enriched with information about sight-seeing and visiting places. Three user roles: **ADMIN**, **SCOUT**, **General USER**.

- **ADMIN:** main controlling point – adds/removes users, makes posts available, deletes/updates posts, removes comments.
- **SCOUT:** collects information about visiting places (costs, travel medium, short history, country representation) and creates post requests for admin to publish.
- **General USER:** views published posts, searches with filters (country, genre, cost), maintains a wishlist (checklist), gets probable cost information, posts comments.
- **Non-registered users** see only a simple home page.

Team Structure & Student IDs

Task	Main Features
Task 1	User Authentication, Registration, Profile, Home Page, Wishlist (CRUD on wishlist)
Task 2	Scout Post Request & Information Submission (full CRUD on post_requests) + Request Changes
Task 3	Admin Dashboard – User Management (add/remove users, verify) + Post & Comment Moderation (full CRUD on users, posts, comments)
Task 4	General USER – Browse Published Posts, Search & Filtering (AJAX), Comments, Probable Cost Estimate

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/scout/user), is_verified, profile_picture, created_at
posts	id, scout_id, title, short_history, country, genre, cost_level (low/medium/high), travel_medium_info, status (pending/approved/rejected), created_at, updated_at
post_requests	id, scout_id, post_data (JSON containing all place info), requested_at, status (pending/approved/rejected)
wishlist	id, user_id, post_id, added_at
comments	id, post_id, user_id, content, created_at
cost_estimates	id, post_id, base_cost, currency, last_updated

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with password_hash() / verified with password_verify()
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- session_start() on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return Content-Type: application/json; handle errors client-side
- File uploads go to public/uploads/ (profile pictures, post images) – validate MIME type + size server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.
4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, post submission, comment).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into main via PR.

Git Flow

- Repository has a **main** branch (protected, no direct pushes).
- Each student creates a **feature branch** from **main** named **feature/taskX-studentID** (e.g., **feature/task1-1839195-3**).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into **main** and merge.
- Final **main** must contain all four feature branches merged with full history.

Balanced Task Breakdown

Task 1 – User Authentication, Registration, Profile, Home Page & Wishlist

CRUD included: Create (registration), Read/Update (profile), Create/Delete (wishlist).

Requirements:

- **Registration** – form for all roles (admin, scout, general user). Collect: name, email, password (≥ 8 chars), role (dropdown). Validate unique email, hash password, set `is_verified = 0` (admin must verify later). Redirect to login with flash message.
- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” checkbox stores secure random token (hashed) in `users.remember_token` and writes 30-day cookie. On subsequent visits PHP reinstates session automatically.

- **Profile Page** (session-gated) – user can update name, email, profile picture (upload to public/uploads/), and change password (require current password). Show success banner on save.
- **Verification Notice** – non-verified users see a message “Your account is pending admin approval” on login and cannot access detailed site features.
- **Logout & Navbar** – logout destroys session and deletes remember cookie. Navbar shows different links based on role and verification status.
- **Home Page** – for **non-registered users**: simple greeting with call to action to register/login. For **logged-in but not verified** users: show pending approval message. For **verified users**: show latest approved posts (summary cards) and a link to full browse.
- **Wishlist (Checklist)** – only verified **General USERS** (role='user' and is_verified=1) can:
 - Add a post to wishlist (AJAX POST to /api/wishlist/add).
 - View wishlist page – list all saved posts with title, country, cost level, and a “Remove” button.
 - Remove from wishlist (AJAX DELETE to /api/wishlist/remove).
 - Uses wishlist table, prepared statements, JSON responses.
- **JS validation** on registration and profile forms (e.g., password match, email format).
- **AJAX** for wishlist add/remove (update UI without reload).

Key outputs: users table populated with all roles; session + “Remember Me” functional; wishlist CRUD fully operational.

Task 2 – Scout Post Request & Information Submission (including Request Changes)

CRUD included: Full CRUD on post_requests (Create, Read, Update, Delete) + image upload.

Scope: Medium – similar to Task 1; scouts can manage their own requests and request changes to approved posts.

Requirements:

- **Scout Gate** – only users with role = 'scout' and is_verified = 1 can access scout pages.
- **Create Post Request** – form to submit information about a visiting place:
 - Title, short history, country representation (e.g., cultural significance), genre (dropdown: beach, mountain, city, historical, etc.), cost_level (low/medium/high), travel_medium_info (e.g., flight, train, bus).
 - Upload images (optional, store in public/uploads/posts/).
 - Data is saved into post_requests table with status = 'pending'.
- **My Requests Page** – list all post requests made by the logged-in scout. Each row shows status, edit/delete buttons (only if status is pending).

- **Edit** – pre-fill form, update the `post_requests` record (AJAX or regular POST).
 - **Delete** – remove the request entirely (AJAX DELETE with confirmation).
- **View Approved Posts** – scout can see which of their submissions became approved (read-only from `posts` table).
- **Request Changes for Approved Posts** – for an approved post (in `posts` table), scout can create a new post request that references the original post ID (e.g., add a field `original_post_id` in `post_requests`). This request will be reviewed by admin as a change request.
- **PHP & JS validation** on all forms (e.g., required fields, image size/type).
- **AJAX** for delete/edit form submission to avoid page reload.

Key outputs: `post_requests` fully manageable; scouts can submit, edit, delete requests, and request changes to published posts.

Task 3 – Admin Dashboard: User Management + Post & Comment Moderation

CRUD included: Delete/Update on `users` (add/remove users, verify), Delete/Update on `posts` (approve, edit, delete), Delete on comments.

Scope: Medium – admin has three sub-modules but each is straightforward; includes adding new users manually.

Requirements:

- **Admin Gate** – every admin page checks `$_SESSION['role'] === 'admin'`; redirect others.
- **Dashboard Summary** – counts of total users (by role), pending post requests, total posts, total comments.
- **User Management** – list all users (scout & general users). Admin can:
 - **Add new user** – form to create a user (name, email, password, role) – sets `is_verified = 1` immediately or 0 as needed.
 - **Verify/unverify** – toggle `is_verified` (verified users get full access).
 - **Delete user** – cascade remove their posts, requests, wishlist, comments.
 - (Optional) Change role.
- **Post Moderation** – list all `post_requests` (pending) and all `posts` (approved/rejected).
 - **Approve a pending request** – move data from `post_requests` to `posts` (create a new post with `status='approved'`), delete the request.
 - **Reject request** – delete or mark rejected with reason.
 - **Edit any approved post** – directly update `posts` table (title, history, country, etc.).
 - **Delete any post** – remove post and its associated comments, wishlist entries.
- **Comment Moderation** – list all comments (with post title and commenter name). Admin can delete any comment (AJAX or POST).

- **AJAX** – for toggling user verification, approving a post request (inline update), deleting comments.
- **PHP validation** on all admin actions (e.g., ensure email unique when adding user, prevent deleting own admin account).

Key outputs: Admin fully controls users, posts, and comments; CRUD operations across three tables; admin can add users manually.

Task 4 – General USER: Browse Published Posts, Search & Filtering (AJAX), Comments, Probable Cost Estimate

CRUD included: Create/Delete (comments by user), Read (search/filter/post details), plus cost calculation logic.

Scope: Medium – interactive features: search, filters, comments, cost estimate.

Requirements:

- **User Gate** – only verified general users (`role='user'` and `is_verified=1`) can post comments and save wishlist (wishlist already in Task1). However, browsing and searching should be accessible to all verified users (including scouts and admin? But this task focuses on General USER experience). We'll restrict comment posting and cost estimate usage to General USERS.
- **Browse Published Posts** – display all approved posts (`status='approved'`). Each post card shows: title, country, genre, `cost_level`, short history snippet, and a “Read more” link/button to detail page.
- **View Detailed Information** – on post detail page, show full short history, country representation, travel medium info, and images.
- **Search & Filtering (AJAX)** –
 - **Search box** (live search) – on keystroke, call `GET /api/posts/search?q=...` returns matching posts (title or country).
 - **Filters** – country dropdown, genre checkboxes (or multiselect), `cost_level` radio (low/medium/high).
 - AJAX `GET /api/posts/filter` with query parameters (country, genre, cost) returns JSON; JS dynamically updates post grid without page reload.
- **Comments** – on post detail page:
 - View existing comments (show reviewer name, comment, date).
 - Logged-in General USER sees a form: name (pre-filled from profile, editable), comment field.
 - Submit comment (AJAX `POST` to `/api/comments/add`). PHP validates non-empty, saves to `comments` with `user_id`.
 - User can delete **their own** comment (AJAX `DELETE` to `/api/comments/{id}`).
 - Comments appear instantly via JS.

- **Probable Cost Information –**

- Show base cost from `cost_estimates` table (or derive from `cost_level` using a mapping: low=500, *medium* =1500, high=\$3000).
- Provide a small calculator: number of travelers (1-10), number of days → total = $\text{base_cost} * \text{travelers} * \text{days} / 7$ (or similar). Use client-side JS or AJAX to display estimated total.
- **JS validation** on comment form (non-empty, max length) and on cost calculator (positive integers).
- **PHP validation** on comment submission (check user is verified, non-empty content, prevent XSS).

Key outputs: General users have full interactive experience: browse, search with dynamic filters, post/delete comments, get cost estimates. All AJAX endpoints return JSON.